

Virtual Lab Experiment - 5

End-to-End DevOps CI/CD Pipeline

Aim

To design and implement a complete DevOps pipeline where infrastructure is provisioned using Terraform, servers are configured using Ansible, application containers are built using Docker, deployed on Kubernetes, and automated using GitHub Actions CI/CD.

Scenario (Real-Time Use Case)

A startup wants to deploy a web application with **zero manual intervention**.

The requirements are:

- Infrastructure should be **created automatically**
- Application should be **containerized**
- Deployment should be **scalable**
- Any code change should **trigger automatic deployment**

Solution Stack:

Requirement	Tool
Version Control	Git
CI/CD	GitHub Actions
Infrastructure Provisioning	Terraform
Configuration Management	Ansible
Containerization	Docker
Orchestration	Kubernetes

Architecture Overview

1. Developer pushes code to GitHub
2. GitHub Actions pipeline triggers automatically
3. Terraform creates VM / cloud infrastructure
4. Ansible installs Docker & Kubernetes dependencies
5. Docker builds application image
6. Kubernetes deploys the containerized application

Tools & Technologies

- Git & GitHub
- GitHub Actions
- Terraform
- Ansible
- Docker
- Kubernetes (Minikube / Kubeadm)
- Linux (Ubuntu / Amazon Linux)

System Requirements

- OS: Linux / Windows / macOS
- RAM: Minimum 8 GB
- GitHub account

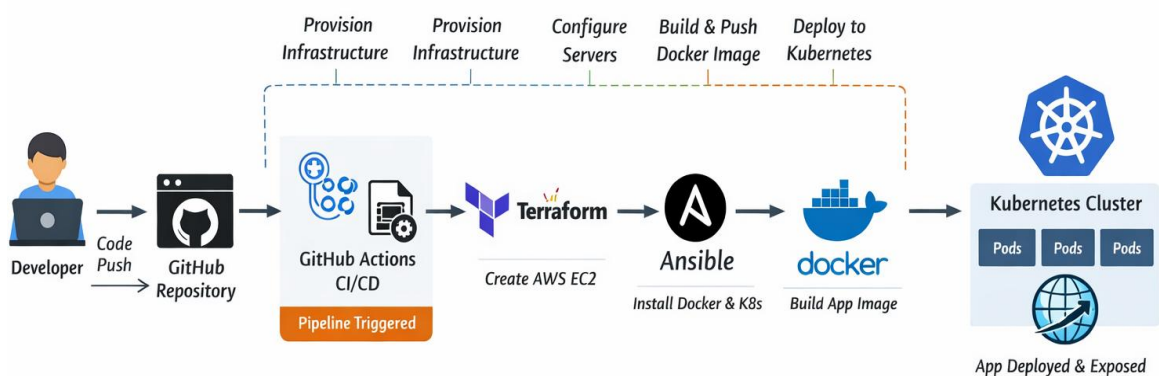
- Docker & Minikube installed (for local lab)

Project Directory Structure

```
devops-virtual-lab/
├── app/
│   ├── index.html
│   └── Dockerfile
├── terraform/
│   └── main.tf
├── ansible/
│   ├── inventory.ini
│   └── setup.yml
├── k8s/
│   ├── deployment.yml
│   └── service.yml
├── .github/workflows/
│   └── cicd.yml
```

Step-by-Step Implementation

End-to-End DevOps CI/CD Pipeline



PART A – Git & Application Setup

Step 1: Initialize Git Repository

```
git init
git branch -M main
```

Step 2: Create Web Application

```
mkdir app
nano app/index.html
<!DOCTYPE html>
<html>
<body>
<h1>DevOps CI/CD Virtual Lab</h1>
<p>Deployed using Git, Terraform, Ansible, Docker & Kubernetes</p>
</body>
</html>
```

PART B – Docker Implementation

Step 3: Create Dockerfile

```
nano app/Dockerfile
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
```

Step 4: Test Docker Locally

```
docker build -t devops-lab-app .
docker run -d -p 8080:80 devops-lab-app
```

✓ Verify at <http://localhost:8080>

PART C – Terraform (Infrastructure as Code)

Step 5: Create Terraform Configuration

```
mkdir terraform
nano terraform/main.tf
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "devops_server" {
  ami          = "ami-0c02fb55956c7d316"
  instance_type = "t2.micro"
  key_name     = "my-key"

  tags = {
    Name = "DevOps-Lab-Server"
  }
}
```

Step 6: Execute Terraform

```
terraform init
terraform plan
terraform apply -auto-approve
```

✓ EC2 instance created automatically

PART D – Ansible (Configuration Management)

Step 7: Create Ansible Inventory

```
mkdir ansible
nano ansible/inventory.ini
[servers]
server1 ansible_host=<PUBLIC_IP>

[servers:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=my-key.pem
```

Step 8: Create Ansible Playbook

```
nano ansible/setup.yml
- name: Configure Server
  hosts: servers
  become: yes
  tasks:
    - name: Install Docker
      yum:
        name: docker
        state: present

    - name: Start Docker
      service:
        name: docker
        state: started
        enabled: yes
```

Step 9: Run Ansible Playbook

```
ansible-playbook -i inventory.ini setup.yml
```

✓ Docker installed automatically

PART E – Kubernetes Deployment

Step 10: Create Kubernetes Deployment

```
mkdir k8s
nano k8s/deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: devops-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: devops
  template:
    metadata:
      labels:
```

```
  app: devops
spec:
  containers:
  - name: devops-container
    image: devops-lab-app
  ports:
  - containerPort: 80
```

Step 11: Create Service

```
nano k8s/service.yml
apiVersion: v1
kind: Service
metadata:
  name: devops-service
spec:
  type: NodePort
  selector:
    app: devops
  ports:
  - port: 80
    targetPort: 80
```

Step 12: Deploy to Kubernetes

```
kubectl apply -f deployment.yml
kubectl apply -f service.yml
```

PART F – GitHub Actions (CI/CD Automation)

Step 13: Create GitHub Actions Workflow

```
mkdir -p .github/workflows
nano .github/workflows/cicd.yml
name: DevOps CI/CD Pipeline

on:
  push:
    branches: [ "main" ]

jobs:
  build-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v3

      - name: Build Docker Image
        run: docker build -t devops-lab-app ./app

      - name: Deploy to Kubernetes
```

```
run: /  
kubectrl apply -f k8s/deployment.yml  
kubectrl apply -f k8s/service.yml
```

Step 14: Push Code to GitHub

```
git add .  
git commit -m "End-to-End DevOps Virtual Lab"  
git push origin main
```

✓ CI/CD pipeline triggers automatically

Observations

- Infrastructure created automatically using Terraform
- Server configured using Ansible
- Application containerized using Docker
- Application deployed and scaled using Kubernetes
- CI/CD automated using GitHub Actions

Result

An end-to-end automated DevOps pipeline was successfully implemented using Git, GitHub Actions, Terraform, Ansible, Docker, and Kubernetes.

VIVA VOCE QUESTIONS

SECTION A – GIT (Version Control System)

Intermediate Level

1. Difference between git pull and git fetch
2. What is branching in Git?
3. What is the default branch in Git?
4. Difference between git merge and git rebase
5. What is a conflict in Git?
6. How do you resolve merge conflicts?
7. What is HEAD in Git?

Advanced / Tricky

8. What happens internally when you run git commit?
9. How does Git track file changes?
10. What is the difference between soft, mixed, and hard reset?
11. What is a detached HEAD state?
12. How is Git useful in CI/CD pipelines?

SECTION B – GitHub

Intermediate Level

1. Difference between fork and clone
2. What are issues in GitHub?
3. What is a GitHub organization?
4. What is a webhook?
5. What are GitHub secrets?

Advanced / Practical

6. How does GitHub help in team collaboration?
7. What is code review in GitHub?
8. How is GitHub integrated with CI/CD tools?
9. What is the role of GitHub in DevOps?

SECTION C – GitHub Actions (CI/CD)

Intermediate Level

1. What is a job in GitHub Actions?
2. What is a step in GitHub Actions?
3. What is a runner?
4. Difference between self-hosted runner and GitHub-hosted runner
5. What is actions/checkout?

Advanced / Scenario-Based

6. How do you pass secrets in GitHub Actions?
7. How does GitHub Actions automate Docker builds?
8. How can GitHub Actions deploy to Kubernetes?
9. What happens if a workflow fails?
10. Compare GitHub Actions with Jenkins

SECTION D – Docker

Basic Level

1. What is Docker?
2. What problem does Docker solve?
3. What is a container?
4. Difference between VM and Docker container
5. What is a Docker image?
6. What is Docker Hub?

Intermediate Level

7. What is a Dockerfile?
8. Difference between docker run and docker start
9. What is the use of EXPOSE in Dockerfile?
10. What is a Docker volume?
11. What is the Docker lifecycle?

Advanced / Practical

12. How does Docker help in microservices?
13. Difference between image and container
14. How does Docker improve deployment speed?
15. What happens when you run docker build?
16. Can Docker containers communicate with each other? How?

SECTION E – Kubernetes

Basic Level

1. What is Kubernetes?
2. Why is Kubernetes needed when Docker exists?
3. What is a Kubernetes cluster?
4. What is a Pod?
5. What is kubectl?

Intermediate Level

6. Difference between Pod and Deployment
7. What is a Service in Kubernetes?
8. What are the types of Kubernetes services?
9. What is NodePort?
10. What is a ReplicaSet?

Advanced / Scenario-Based

11. How does Kubernetes achieve high availability?
12. What happens if a Pod crashes?
13. How does Kubernetes scale applications?
14. Difference between Docker Swarm and Kubernetes
15. What is Minikube and why is it used?
16. How is Kubernetes used in production environments?

INTEGRATION & DevOps FLOW QUESTIONS (Very Important)

17. Explain a DevOps workflow using Git, Docker, and Kubernetes
 18. How does GitHub Actions fit into DevOps?
 19. How does a code push trigger deployment?
 20. Explain CI vs CD with an example
 21. How do Docker and Kubernetes work together?
 22. What happens from git push to application deployment?
 23. How is automation achieved in DevOps?
 24. Give a real-time use case for DevOps tools
-